

5.3 Test-Driven Development with AI

Test-driven development (TDD) is a cyclical process that is often described as a TDD cycle. It consists of three phases: red, green, and refactor. The first step is a failed test, as it occurs before the implementation. The red color stands for the failure of the test. You then implement the program logic that ensures the test can run successfully. This turns the test into green. In the final step, you have the opportunity to optimize your code. You carry out refactoring in which the interface to the outside world remains the same and nothing changes in terms of functionality. [Chapter 4](#) looks at how AI can help you with this and what strategies are available.

In TDD, you don't start directly with the development. Instead, first think about the requirements that your code unit, usually a function, should fulfill, and formulate these in the form of one or more tests. Then, you implement the function. You proceed in small steps, so-called baby steps, and implement the function step by step or, rather, test by test.

In this chapter, we use two different AI tools for TDD, but in real life, you'll rarely do this. This example is rather intended to show how you can get help with the various parts of the development process.

As a concrete example of TDD, we're implementing a function for checking credit card numbers. Such a number is made up of different parts: the first six digits identify the issuing financial institution, while the subsequent digits form the cardholder's account number. The last digits, which are relevant for our example, form the check digit. This is calculated using the Luhn algorithm.

Because we want the AI tool to do the work for us and not keep us unnecessarily busy, we have two tools working together here: ChatGPT writes the test, and GitHub Copilot takes care of the implementation.

5.3.1 Implementing the First Test

Tools like ChatGPT are very useful and usually try to give a comprehensive answer. However, this is more of a hindrance for our current task. To prevent the tool from showing you how to set up your test environment, deliver all the tests and implementation, and run the tests, you need to make the prompt very clear:

Prompt: Assume the environment is already set up and everything is installed. Create a test in vitest using TypeScript for a function called `validateCreditCardNumber` to validate a given credit card number. Use TDD and baby steps as a strategy. Only write the first test; no info on how to run the test is needed.

Sure, let's start with the first step in Test-Driven Development (TDD): writing a test for a nonexistent function `validateCreditCardNumber`.

Step 1: Write the First Test. We'll write a test that checks if the `validateCreditCardNumber` function exists and is callable. Let's create a test file `validateCreditCardNumber.test.ts`:

```
import { describe, it, expect } from 'vitest';
import {
  validateCreditCardNumber
} from './validateCreditCardNumber';

describe('validateCreditCardNumber', () => {
  it('should be a function', () => {
    expect(typeof
validateCreditCardNumber).toBe('function');
  });
});
```

[Explanation of the code]

With this prompt, ChatGPT keeps the response concise and focused. The generated code imports the functions required by Vitest as well as the implementation of the `validateCreditCardNumber` function. The first test only checks whether there really is a function behind this variable. If you save

the code in a file named `validateCreditCardNumber.test.ts`, as suggested by ChatGPT, and then run the test via `npx vitest`, you'll receive an error message indicating that the `validateCreditCardNumber` file doesn't exist. So, in the next step, you create the `validateCreditCardNumber.ts` file.

You can then instruct GitHub Copilot to generate the implementation for you. You can achieve this with the following prompt, for example:

Prompt: Make the following test pass:

[code of the test]

Sure! Here's the code block you can use to make the test pass:

```
export function validateCreditCardNumber(
  cardNumber: string
): boolean {
  // Your implementation here
  return true;
}
```

Running the test with this implementation returns a green result, so it's time for the next test.

GitHub Copilot has enhanced its capabilities to reference other files within a project. In Visual Studio and Visual Studio Code, you can reference specific files in GitHub Copilot Chat by using the `#` symbol followed by the file name. For example, to reference a file named `validateCreditCardNumber.ts`, you would use `#validateCreditCardNumber.ts`.

Additionally, GitHub has introduced Copilot Workspace, a Copilot-native development environment designed to assist developers with tasks ranging from brainstorming to coding and testing. Copilot Workspace enables AI-assisted development within a repository, allowing for programming tasks embedded in a larger context.

5.3.2 The Second Test

If the first test is still in the context window of the ChatGPT session, you can start from the previous prompt and ask ChatGPT to implement the next test:

Prompt: Create the next test.

Step 2: Write the Next Test. We will now write a test to check if the `validateCreditCardNumber` function correctly validates a valid credit card number. Here's an updated version of `validateCreditCardNumber.test.ts` with the next test:

```
import { describe, it, expect } from 'vitest';
import {
  validateCreditCardNumber
} from './validateCreditCardNumber';

describe('validateCreditCardNumber', () => {
  (existing code)

  it('should return true for a valid credit card
number', () => {
    // Example of a valid Visa card number
    const validCreditCardNumber = '4111111111111111';

    expect(validateCreditCardNumber(validCreditCardNumber))
      .toBe(true);
  });
});
```

[explanation of the code]

The second test already includes significantly more framework conditions. It transfers a credit card number as a character string and expects the `true` value as the return value. If you run the tests, you'll find that both are already successfully executed without changing the source code. The reason for this is that the function always returns the `true` value. So, for now, no

further changes to the code are necessary, but the next test is due immediately.

5.3.3 The Third Test

Now, it's ChatGPT's turn again with the next test. Due to the existing context, this prompt is also kept very simple:

Prompt: Create the next test.

Sure, let's proceed by writing the next test for the `validateCreditCardNumber` function. This test will check if the function correctly identifies an invalid credit card number. Here's an updated version of `validateCreditCardNumber.test.ts` with the next test:

```
import { describe, it, expect } from 'vitest';
import {
  validateCreditCardNumber
} from './validateCreditCardNumber';

describe('validateCreditCardNumber', () => {
  (existing code)

  it(
    'should return false for an invalid credit card number',
    () => {
      // Example of an invalid credit card number
      const invalidCreditCardNumber =
        '1234567890123456';

      expect(validateCreditCardNumber(invalidCreditCardNumber))
        .toBe(false);
    });
});
```

[explanation of the code]

This third test is a classic negative test. It allows you to test whether the function also handles incorrect entries correctly. This also forces you to

adapt the implementation of the function so that the algorithm is correct. In the prompt, you pass the three tests to GitHub Copilot and instruct it to change the code accordingly.

Prompt: Change the implementation, so that the following three tests pass:

[code of the test]

```
// Implementation of validateCreditCardNumber function
export function validateCreditCardNumber(
  cardNumber: string
): boolean {
  // Remove any non-digit characters from the card
  number
  const cleanedCardNumber = cardNumber.replace(/\D/g,
  "");

  // Check if the cleaned card number is empty or not
  if (cleanedCardNumber.length === 0) {
    return false;
  }

  // Use Luhn algorithm to validate the card number
  let sum = 0;
  let double = false;

  for (let i = cleanedCardNumber.length - 1; i >= 0;
  i--) {
    let digit = parseInt(cleanedCardNumber.charAt(i),
    10);

    if (double) {
      digit *= 2;
      if (digit > 9) {
        digit -= 9;
      }
    }

    sum += digit;
    double = !double;
  }
}
```

```
    return sum % 10 === 0;
}
```

Copilot completed the task with flying colors and wrote the code in such a way that all three tests could be passed successfully. The individual steps in the function are provided with descriptive comments so that you can easily follow the code.

The step from the empty function to the final implementation doesn't correspond to the idea of TDD. Further intermediate steps would have been advisable here. In this case, you should have been much stricter in the design of the tests and also in the prompting for Copilot and guided the AI tool better. Again, the following also applies here: the more precisely you formulate your prompts, the better you can control the result.

5.3.4 Generating Test Data

If you're still skeptical as to whether the implementation really works, you can have another test generated that checks a whole range of credit card numbers. For such a mass test, you can use the `it.each` method to test a range of values and still keep your code compact. The corresponding prompt could read as follows:

Prompt: Create 10 valid and 10 invalid credit card numbers, and use `it.each` to verify if the `validateCreditCardNumber` function works correctly.

Sure, let's enhance our tests using `it.each` to validate multiple credit card numbers at once. We'll create 10 valid and 10 invalid credit card numbers and use `it.each` to verify if the `validateCreditCardNumber` function works correctly.

Here's the updated `validateCreditCardNumber.test.ts`:

```
import { describe, it, expect } from 'vitest';
import {
  validateCreditCardNumber
} from './validateCreditCardNumber';
```

```

const validCreditCardNumbers =
  [... validCreditCardNumbers ... ];

const invalidCreditCardNumbers =
  [ ... invalidCreditCardNumbers ... ];

describe('validateCreditCardNumber', () => {
  it('should be a function', () => {
    expect(typeof
validateCreditCardNumber).toBe('function');
  });

  it.each(validCreditCardNumbers)(
    'should return true for a valid credit card
number: %s',
    (cardNumber) => {

expect(validateCreditCardNumber(cardNumber)).toBe(true);
    });

  it.each(invalidCreditCardNumbers)(
    'should return false for an invalid credit card
number: %s',
    (cardNumber) => {

expect(validateCreditCardNumber(cardNumber)).toBe(false);
    });
});

```

[explanation of the code]

Due to the existing context, ChatGPT has understood this prompt as an improvement, that is, refactoring of the existing tests. The implementation of `validateCreditCardNumber` remains untouched, but the code of the tests changes in this case. Generally, refactoring of the tests is also permitted. In this example, you define two arrays, one with valid credit card numbers and one with invalid ones.

The first test as to whether `validateCreditCardNumber` is a function remains unchanged. For the valid and invalid credit card numbers, you use the `it.each` function to generate a separate test for each value. The `it.each`

function uses placeholders in the description so that the respective value gets inserted in the output.

In the next section, you'll learn how to deal with external dependencies in your tests and how AI tools can help you with this.